

Technical Architecture

The document viewer as a product a third-party common data environment embeds.

Status. The rendering core is built and boundary-enforced, and so — since this document was last issued — is the integration surface: the `cde.viewer.v1` embed protocol is implemented on both sides and tested against an independent host, the backend carries the `frame-ancestors allow-list` that used to refuse every frame, and **the image now serves the embed document itself**, which is what closed the gap this document reported last time. The demo host now drives the whole protocol against it end to end. What has not happened is a real CDE integrating it. Section 13 states exactly what is left — read it before quoting anything here to a customer.

Scope. The viewer as described by ADR 12 (`cde-platform, docs/adr/0012-viewer-as-a-standalone-product.md`).

Companions. `viewer-integration-guide.md` for how a host uses it; `viewer-extraction-inventory.md` for what still has to move; and the platform's own `docs/architecture.md`, which remains the authority on tenancy, the ISO 19650 state machine and the request lifecycle.

Contents

1. What this is, and what it is not

It is a browser-side viewer for construction documents — PDF, DXF/DWG drawings, and IFC models — with a markup, measurement, redaction and signature layer over the top, plus the server-side conversion needed to turn formats a browser cannot read into ones it can.

It is not a document management system. It does not own documents, users, projects, permissions or an audit trail. The host CDE owns all of those, and the viewer's job is to borrow a document briefly, render it, and hand back what the user drew on it.

That distinction is the whole architecture. Every design decision below follows from “the bytes belong to someone else”.

2. Three layers, and the one that matters

HOST CDE	EXCHANGE	VIEWER PRODUCT
Their web interface where the viewer appears	1. mount the viewer → ✓ BUILT iframe + cde.viewer.v1	/embed route no session, no auth guard
Their identity provider Entra / Okta / own	2. identify the user → ✓ BUILT host.init.identity — 3 untrusted fields	viewer-core 15 files · 6 services · 6 components
Their document store SharePoint / S3 / Blob	3. hand over the document → ✓ BUILT host.init.document, or POST /api/conversions	Conversion pipeline fetch → scan → render
Their markup store they stamp the author	← 4. return what was drawn ✓ BUILT 3 markup events + 1 operation pair	host-channel origin-checked in both directions
	⚠ NOT YET DEPLOYABLE the allow-list exists; nothing serves the embed document	

Figure 1 — All four exchanges now have code on both sides, checked against each other by a demo host written in plain JavaScript. The frame-ancestors allow-list the backend refused to have now exists — `cde.web.embed-parent-origins`, closed until a deployment names a host. What is still missing is a tier that serves the `/embed` document at all: the backend image carries no frontend, and the Kubernetes manifests route everything to the backend.

src/viewer-core/ is the product. 3,220 lines of production TypeScript across 15 source files, with 8 spec files beside them. Nothing in it imports anything from the application — not a service, not a model, not an environment constant. When the separate repository exists, this directory is **copied, not untangled**.

It is now shaped as a library rather than a directory: `package.json`, `ng-package.json`, `tsconfig.lib.json` and a curated `index.ts`, registered in `angular.json` as a second project built by `@angular/build:ng-packagr`. The name is `@cde/viewer-core` and the licence field reads `UNLICENSED` — deliberately, so an accidental `npm publish` fails rather than quietly shipping to a public registry with no licence position. Removing that marker is a commercial decision, not a cleanup.

The application shell above it is this platform's own consumer of the viewer. It is not part of the product; it is the first integrator, and useful precisely because an awkward contract will be awkward for us first. Since the embed shipped there is a **second** consumer — `demo/`, a host application in plain JavaScript — and it is the more honest of the two, because it is the side an integrator actually writes.

2.1 The boundary is asserted, not intended

`viewer-core.boundary.spec.ts` reads every `./*.ts` in the directory through Vite's `import.meta.glob` and fails on four conditions.

Assertion	Why it exists
At least 6 source files present	A renamed directory would otherwise make every check below vacuously true
Nothing imports <code>app/</code> , <code>core/</code> , <code>features/</code> , <code>shared/</code> , <code>environments/</code>	The dependency that breaks extraction
Nothing reaches above the directory except <code>../testing/</code> from a spec	Catches the same thing if a folder is renamed
No production file reaches outside at all	Stops the test-helper exemption becoming a hole

Table 1 — The four boundary assertions.

One `inject(AuthService)` added in a hurry turns a copy into a migration, and **nothing else in the build would notice** — the application compiles perfectly well with the dependency pointing the wrong way. That is why this is a test and not a convention.

Components sit flat beside the services rather than in a `components/` subdirectory. The glob is `./*.ts` and the rule forbids every `../ import`, so a nested component reaching back for `../viewer-state.service` would both escape the glob and trip the rule it escaped.

3. viewer-core

3.1 Services

Service	Lines	Responsibility
<code>viewer-state.service</code>	375	The single source of truth. Signals for document, page, zoom, rotation, active tool, selection, sidebar tab, and the version-commit token that makes panels reload after a server-side operation
<code>markup-engine.service</code>	565	All drawing: pointer events (mouse and touch), shape creation, SVG rendering, hit testing
<code>pdf-engine.service</code>	201	<code>pdfjs-dist</code> wrapper — document loading, page rendering to canvas, text-layer extraction
<code>measurement.service</code>	165	Scale calibration and the geometry behind length, area and radius. <code>unitsPerPixel === 1</code> with unit <code>px</code> means uncalibrated
<code>outline.service</code>	147	A PDF's bookmarks and its link annotations
<code>drawing-search.service</code>	129	Text search across a converted drawing's labels

Table 2 — The six server-independent services.

3.2 Components

cad-viewer (the DXF/DWG canvas), ifc-tree (the model hierarchy), outline-panel, page-links, tool-rail, and icon — the last being the product's whole icon set as stroke-only 24×24 path data, so one definition sits on a light rail, a dark header and an accent-filled button without variants.

cad-viewer.component.ts is 629 lines, over the §3.3 limits of 400 per file and 200 per component. Recorded rather than hidden: it is known debt, it predates the extraction, and splitting a canvas component wants a test suite that can actually run first.

3.3 What is deliberately not here

The 15 components still in the application are not there by accident. Thirteen import the platform's HTTP services; two (viewer.component, viewer3d.component) inject ViewerService, which calls /api/viewer/{id} and the annotation endpoints.

ADR 12's open step 4 is now closed. ADR 14 decided the rule: reads and computations are viewer-side, and anything that changes the document is a callback into the host. That covers seventeen operations, and three of them — document.sign, version.create, version.restore — never become viewer-side whatever else changes, because a signature over a copy the host has since replaced is worse than no signature. What remains is the work of moving these components, not the question of where they go.

4. The embed surface

New since the last issue of this document, and the reason its status line changed. The contract ADR 14 settled is now code on both sides.

src/app/features/embed/ — 1,128 lines of production TypeScript across six files, with 811 lines of spec beside them. It is deliberately **not** in viewer-core: the protocol is how a host talks to a deployment of the viewer, so it belongs to the application that is deployed, while viewer-core stays the part that knows no backend exists.

File	What it does
embed-protocol.ts	The envelope, the message union, and the validators. No Angular, no DOM — it is the contract as types plus the predicates that enforce them
host-channel.service.ts	The postMessage boundary. Every inbound message is checked in a fixed order: origin, then event.source, then protocol envelope, then direction
embed-session.service.ts	The session state machine — ready, awaiting init, loaded, failed — and the document-open path
embed-page.component.ts	The rendering surface and the markup overlay
embed-viewer.component.ts	The /embed route component. No auth guard, deliberately — the viewer authenticates nobody
markup-wire-format.ts	Translation between the viewer's ShapeData and the protocol's Markup — the only part of the session with no session state, so it tests against the two representations directly

Table 3 — The embed implementation.

Fourteen message types run viewer → host and six run host → viewer. Four of the fourteen are lifecycle events — `viewer.opened`, `viewer.unloaded`, `viewer.markupLoaded` and `viewer.pageRendered` — added after the first integrations asked how to record what a reader did. None is required: a host can ignore every one and still open documents and collect markup. Adding them was additive within v1, which is the compatibility promise in §10 of the protocol doing the job it exists for.

`viewer.pageRendered` is deduplicated per document, and that is the whole point of it. Zooming repaints a page; it is not a second reading of it. An undeduplicated stream would inflate a “pages read” figure by however many times the user changed zoom, and a host building that metric would have no way to tell. The viewer deduplicates so that every host does not have to, and so that the ones that forget are not silently wrong.

4.1 Five rules, and the order they are checked in

§5.13.8 and the protocol’s own §3 give five rules that are not negotiable: never post to `'*`, check the origin on every message rather than once, check `event.source` as well as the origin, validate in both directions, and never put a credential in a message. The first four are enforced in `host-channel.service.ts`, in that order, and a message failing any of them is dropped with the reason recorded.

The order is load-bearing, and so is checking `event.source`. Origin alone is not enough: any frame on the host’s page — an advertisement, a third-party widget — posts from the host’s origin. Without the source check a sibling frame can drive the viewer, and nothing about the message would look wrong.

4.2 Two implementations, checked against each other

`demo/public/host-protocol.js` is an independent implementation of the host half — 237 lines of plain JavaScript, no framework, no build step, no dependency on anything in this repository. `protocol-conversation.spec.ts` then runs the **real** viewer channel against the **real** demo host through two fake windows, and asserts the conversation rather than either side’s internals.

That is worth more than either side tested against a mock. A mock encodes the same assumption the implementation makes, so the two agree by construction; two implementations written from the specification disagree wherever the specification was ambiguous, which is exactly where an integrator would have disagreed with us.

A green test proves nothing until the thing it guards is broken on purpose. The `host-trusts-the-author` rule passed its first test while broken, because the viewer never sends an author and the test asserted an outcome rather than forging the field. The test that works posts a `viewer.markupCreated` carrying `author: "Someone Else"` and asserts the host stores the name from its own session instead.

Eight Playwright tests drive the demo in a real browser against a stub viewer on a third origin. They earn their keep: three defects no unit test could reach were found there — a captured `contentWindow` going stale across navigation, an `author display` rule beating the user agent’s `[hidden] { display: none }` so a placeholder swallowed every click, and the page and the server disagreeing about the viewer’s origin.

4.3 What the embed cannot open

Only PDF renders in an embedded frame today. `embed-session.service.ts` treats `application/pdf` as directly renderable and refuses everything else with a 415 `conversion-required` problem naming the format. The conversion service in section 6 exists and works — the embed path simply does not call it yet. Office and IFC are

therefore supported by the product and **not** supported through the embed, which is a distinction worth stating before a customer discovers it.

5. Three rendering pipelines

PDF is rendered in the browser by pdfjs-dist 6. Text layer extracted for search and redaction; markup drawn as SVG in a layer above the canvas so it scales with zoom without re-rasterising.

DXF/DWG cannot be rendered by the browser at all. The converter turns a DXF into SVG server-side; DWG is converted to DXF first. The result is an SVG the browser renders directly, with an invisible text layer over it for search — the same trick the PDF viewer uses, for the same reason.

IFC is rendered with three 0.185 as WebGL geometry, with ifc-tree carrying the model hierarchy alongside it. **The tree is the primary interface and the canvas is the visual layer over it**, not the other way round — a WebGL canvas cannot be made WCAG-conformant on its own, and §1A.4 requires an equivalent accessible route to the same information. Building the tree first is what makes that true rather than aspirational.

5.1 The same drawing, rendered twice, on purpose

A DXF is converted twice, by two different paths, and the reason is not obvious enough to leave undocumented.

For the viewer, geometry is drawn as SVG paths and the text is emitted as `<text>` with `fill="none"` — invisible, positioned, selectable. The browser draws the glyphs from the geometry; the invisible layer exists only so search and selection have something to hit.

For PDF export, that trick is inverted. LibreOffice discards SVG text it cannot see, so an invisible layer would produce a PDF with no extractable text at all. The print render therefore draws **no** glyph geometry (`TextPolicy.IGNORE`) and emits **visible** `<text>`, which LibreOffice turns into real, searchable PDF text.

The consequence that bites: calibration must measure what was actually drawn. The viewer render includes glyph paths, so it measures the whole layout; the print render draws no text, so it measures geometry only. Using the wrong set silently shifts every label on the sheet.

A related trap, recorded because it cost a day: rendering white-on-white passes every test that checks the PDF exists, has pages, and contains text. Only rasterising the output and measuring ink coverage catches it, and that check now lives in the converter's test suite.

6. Why conversion is server-side and stays there

Office documents go through LibreOffice, DWG through the ODA File Converter, OCR through Tesseract, DXF through ezdxf. None of these run in a browser, and all of them are heavy native parsers on untrusted input — §5.13.10 requires them in a sandboxed, resource-limited, network-isolated worker with a timeout, out of process.

This is the constraint that rules out the otherwise-attractive design where the browser fetches straight from the customer's storage and we never see the file: **that architecture cannot render a .docx at all.**

6.1 DWG — resolved by removing the encumbered binary

Changed since the last issue. Two tools can turn binary DWG into DXF, and they had opposite licence problems.

	LibreDWG dwg2dxf	ODA File Converter
Licence	GPL-3.0	Proprietary
In the image	No longer — removed	No — the operator supplies it
The problem it had	We shipped it and owed every recipient corresponding source we did not provide, so any distribution of the image was a breach	We cannot ship it, so a customer install has no DWG unless they obtain one themselves

Table 4 — Two converters, opposite licence problems — and how the first one was ended.

ADR 13 was resolved on 2026-09-08 by deleting the problem rather than answering it. LibreDWG is gone from the converter image, so nothing encumbered is distributed and the licence finding closes by deletion. The image now contains **no DWG reader at all**; DWG requires an operator-supplied ODA File Converter, mounted into the container and pointed at by ODA_PATH.

That is a smaller product and a defensible one. It also costs nothing in fidelity — ODA is the reference implementation, and the pipeline was designed around it before LibreDWG was added as a fallback. Every other format is unaffected: DXF renders through ezdxf with no external tool, and Office, PDF and IFC never touched LibreDWG. The counsel questions become live again only if bundled DWG is ever wanted.

A supplied ODA is fully supported: discovered from a directory or a binary, launched inside a virtual framebuffer (it is a Qt application and opens a display even converting from the command line), probed once at startup, and reported as `odaRunnable` distinctly from `odaInstalled` — a mount missing its libraries or its execute bit is present and useless.

`libredwgInstalled` remains in the status response, always false, marked deprecated with a sunset of 2027-04-01. §3.4 forbids removing a field inside an API version without at least six months' notice, and a client branching on it reads the truth — the tool is not installed.

7. Document ingress

The server-side half of exchange 3, and the one an integrator can use without embedding anything. The host mints a short-lived URL with its own credentials and posts it; the viewer fetches once, converts, and discards.

#	STAGE	WHAT HAPPENS
1	Host mints a link	Short-lived, single-object, your credentials. We never hold them.
2	Submit	POST /api/conversions { sourceUrl, targetFormat } — 202 + Location header, under one second
3	Destination policy	Resolves the hostname and validates the ADDRESS, not the string. Refuses loopback, link-local, 169.254.169.254, RFC 1918, .internal. Redirects are NOT followed.
4	Bounded fetch	Size cap enforced during the stream, not after. Connect

#	STAGE	WHAT HAPPENS
		and read timeouts.
5	Quarantine + scan	Magic-byte type check, then ClamAV. Not referenced by the application until clean.
6	Convert	Sandboxed, resource-limited, network-isolated worker, out of process.
7	Poll	GET /api/conversions/{jobId} — PENDING, RUNNING, then SUCCEEDED / FAILED / CANCELLED (terminal)
8	Collect	GET /api/conversions/{jobId}/content — streamed

Figure 2 — The conversion API, end to end. Stages 1, 7 and 8 are yours; 2 to 6 are ours. Always send an Idempotency-Key so a timed-out retry returns the same job rather than converting twice. An embedded viewer opening a PDF skips this path entirely — the browser fetches your URL directly and no server of ours sees the file.

A refused destination is a **422**, and a job belonging to another tenant is a **404** rather than a 403 — invisible, not forbidden. Every operation requires `document:convert`.

The viewer never holds a customer credential and never learns which storage platform the URL points at — SharePoint, S3, Azure Blob and GCS all reduce to the same code path. That property is what makes “integrates with any CDE” a design rather than a slogan.

The source URL is never stored. It is a bearer credential with a short life; persisting it would turn the job table into a credential store. Only its host is kept on the job record.

The address is checked twice, deliberately: cheaply on submission so an obviously wrong value fails fast with a clear message, and again against the resolved address at fetch time, which is the check that actually decides. No pattern can validate what a name will resolve to.

7.1 The gap under this, narrowed but not closed

The two halves now meet for one format. The embed opens a PDF by fetching the host’s URL directly from the browser — no server of ours is involved, which is the best possible answer for the one format that needs no conversion.

For every other format the halves still do not meet. `/api/conversions` works and the embed does not call it, so an embedded viewer asked for a `.docx` or an `.ifc` returns a 415 `conversion-required` rather than converting it. The application’s own front end likewise still reads `/api/viewer/{id}` and `/api/documents/*`, which assume the platform owns the document. Wiring the embed to the conversion service is the single change that turns “PDF only” into the format list in section 6.

8. State

Angular signals throughout, with `ViewerStateService` as the single store. No `NgRx`, no observable soup: the viewer’s state is small, synchronous and local — zoom, page, tool, selection — and a store framework would add indirection without adding capability.

Server-side operations that rewrite the document (redaction, OCR, flatten, form fill, page rearrange) commit a **new version** and bump a reload token. Panels watch the token rather than each other, which is what lets those operations compose: each starts from the previous one’s output rather than from the untouched original.

9. Accessibility architecture

Not a layer applied afterwards. Three decisions are structural.

- **The IFC tree is the primary data interface.** A canvas alone cannot conform.
- **Every drag interaction needs a single-pointer alternative** (SC 2.5.7) — which for a markup tool means every shape must be creatable and movable without a drag. This is a known gap: the callout box cannot currently be dragged at all, because `updateShape` has no callout case.
- **Exports must be tagged and PDF/UA-conformant.** An inaccessible export is a product accessibility failure, and it is the most common gap found in government audits.

10. Security posture

Most of §5 is inherited from the platform and documented there — RLS tenant isolation, the hash-chained audit trail, PBKDF2 password storage, the §5.4 response headers. What the **viewer product** must own itself once distributed is the question this table answers.

Control	Where it lives now	Where it must live
SSRF policy on the ingress	Platform fetch/ package	Travels with the product — it is the product's own attack surface
Upload magic-byte + AV	Platform	Travels
Content Security Policy	Platform response headers; frame-ancestors is an allow-list on the embed route, 'none' everywhere else	Travels with the product. Deployment-level today; per-tenant needs a discriminator the request cannot forge
Tenant isolation	Platform RLS	Does NOT travel. The host owns tenancy; the viewer must not assume it
Audit	Platform hash chain	Host's concern; the viewer emits events, it does not store them

Table 5 — What travels with the product and what does not.

The last two rows matter most in the long run: a distributed viewer that assumes it owns tenancy would be wrong in a way that is expensive to unwind. **The third row is what matters this week.**

10.1 frame-ancestors, and what it did and did not unblock

Done. `cde.web.embed-parent-origins` names the host origins permitted to frame the `/embed` route. Every other route keeps `'none'`, and a test fails if a relaxation lands globally rather than on that route. `X-Frame-Options` is now written per route rather than globally — it has no allow-list form, so on the embed route it could only say `SAMEORIGIN` and a browser honouring it would refuse the frame however the CSP reads.

Closed by default. An empty list yields `frame-ancestors 'none'` — the same policy, byte for byte, that every other route gets. Adding the route does not on its own make anything frameable, which is what made the change safe to merge before anyone had decided which customers may embed.

This is a security change, not a configuration change. `frame-ancestors` is the authorisation decision about who may frame the viewer; `parentOrigin` in the handshake is only addressing — it says where to post, not who is allowed. Confusing the two would produce a viewer that any site could frame as long as it sent the right message.

Values are validated at startup, because **a `frame-ancestors` source a browser cannot parse is not a closed door** — the browser drops the unparseable source and applies what is left, so a typo silently widens the policy instead of breaking visibly. Wildcards, the `null` origin, CSP keywords offered as origins, anything carrying a path, and non-http schemes are all refused by name.

Deployment-level, not per-tenant, and that limit is deliberate. The embed route carries no credential by design, so there is no authenticated principal to derive a tenant from; deriving one from a query parameter or a header would let a caller nominate its own allow-list, which is a wildcard with extra steps. For the viewer as ADR 12 sells it — a product a customer installs — one deployment is one customer, and that is the boundary this draws. Narrowing further needs a discriminator the request cannot forge.

Where a deployment puts the Angular build behind a separate web tier, that tier serves the document and must carry the same header — the backend's copy governs only what the backend answers. The supported way to avoid that split is 10.2.

10.2 Serving the embed document from the image

The allow-list was necessary and not sufficient: until recently nothing in the deployment served `/embed` at all, because the image carried no frontend while `k8s/ingress.yaml` routed every page request to it. ADR 15 closed that. `cde.web.app.path` points the image at a staged Angular build, and the same component then emits both the document and the policy that governs it.

Empty by default. An image with nothing staged serves no pages and answers page routes with 404 — the behaviour before ADR 15, and the right one where a web tier serves the build. The bundle is staged into the backend's build context before the image is built, because the Angular sources live in a sibling repository a Docker context cannot reach.

The document cannot be a static file. §5.4 forbids `style-src 'unsafe-inline'`, and a production build carries inlined critical CSS, module scripts and a small script that promotes the deferred stylesheet — all of which need a nonce matching the header on the same response. So the document is rendered per request, with `Cache-Control: no-store`, and the build stamps a placeholder the server substitutes. Splitting the document from its header across two components is what this design exists to avoid.

Verified by running the demo host against it, end to end. The full journey works: the document opens, three pages render, markup is drawn and stored by the host, the host page is reloaded and the markup is handed back and painted, `goToPage` and `setZoom` drive the viewer, and a capability that was not granted does not render its control. No content blocked, no console errors.

That run found six defects that every unit test had passed over. A policy refusing the build's own stylesheet-promoting script, so the main stylesheet never applied. `/embed` failing dependency injection before rendering anything. A `pdf.js` worker URL that had never resolved on any deployment. A `connect-src` with no way to name an integrator's storage. A service worker serving the entry document from cache, which defeats a per-request nonce by construction. And markup that never rendered in either direction, because the overlay bound `[innerHTML]` to SVG and Angular's sanitiser strips it — while every protocol message stayed correct, so the host's log looked perfect and the page was blank. **Run the demo against a served viewer before believing an integration works.**

11. Performance

§7.1 applies unchanged: every interactive request under a second, bulk work async with a job id returned in under a second. Conversion is bulk by definition.

Frontend budgets — LCP < 2.0 s, INP < 200 ms, CLS < 0.1, initial bundle < 250 KB gzipped — are the ones the viewer can actually breach on its own. The PDF and 3D renderers are the risk: both are lazy-loaded via `dynamic import()` so they never enter the initial bundle.

Large files never touch application memory. A 2 GB IFC streams to object storage and is processed out of process; §7.7’s prohibition on `readAllBytes` over user content is absolute.

12. Build and verification

Angular 22 with TypeScript strict **and** `noUncheckedIndexedAccess`. Third-party JavaScript is bundled, never loaded from a CDN — §5.12 A08, and it is also what makes air-gapped deployment possible.

A note on the current environment, because it affects what any statement about test results is worth: `ng test` and `ng build` require Node $\geq 22.22.3$, and the development container runs 22.22.2. Specs that need Angular TestBed cannot run there at all, and neither can `ng build viewer-core`; `tsc --noEmit`, bare Vitest and Playwright can. **The /embed route has therefore been typechecked and unit-tested but never rendered by Angular in a browser** — the demo’s end-to-end tests run against a stub viewer, not the real one. Any claim of “tests pass” in this repository should say which of those it means.

12.1 Gates

Each of these fails loudly rather than warning, and each exists because the thing it checks is otherwise invisible in a diff.

Gate	What it asserts
check:no-remote-code	Nothing in <code>src/</code> or <code>demo/</code> loads executable code from a CDN. Two roots, each with its own minimum file count, so a wrong path fails instead of passing vacuously
check:attribution	THIRD-PARTY-NOTICES.txt regenerates to exactly what is committed. It regenerates and asks git, rather than checking the file exists — the backend learned that distinction the hard way, with a shipped attribution naming a version it no longer had
check:icons	The application icons and favicon match their generator byte for byte, and the mark stays inside the maskable safe zone
check:samples	The demo’s three sample documents match their generator byte for byte
test:demo	Eight Playwright tests drive the demo host in a real browser

Table 6 — The repository’s own gates.

Four of these five are not run by CI. The platform’s Jenkinsfile runs `npm ci`, `tsc --noEmit`, `check:no-remote-code`, `ng build` and `ng test` against this repository, and nothing else. `check:attribution`, `check:icons`, `check:samples` and `test:demo` exist, pass, and guard nothing until a pipeline stage calls them. A gate nothing runs is documentation.

13. What this architecture does not yet have

Stated plainly, and shorter than it was. The gap between “the viewer works” and “a CDE can integrate it” used to be a set of undecided contracts; it is now a set of unfinished jobs, in rough order of what blocks what.

No document has opened in a host frame on an installed deployment. Section 10.2. The pieces are each verified — the protocol against an independent host, the allow-list against its own tests, the served document against a real browser — and the whole has never run. This is the next thing to do and the thing most likely to surface something nobody predicted.

The library and the application are built from one repository. The image serves the Angular build staged into the backend’s context, which works and means the frontend is versioned with the backend that carries it. A customer wanting the viewer without the platform still has no artefact of their own; that waits on viewer-core being published, below.

The embed opens PDF and nothing else. Section 4.3. The conversion service exists; the embed path does not call it.

Collaboration has no identity in an embed. Live cursors and presence ride a STOMP socket authenticated by the page session, and an embedded viewer has no session. ADR 14 accepted this as the loose end that “no identity” leaves; it is unsolved rather than decided.

THE PLATFORM’S OWN API — unchanged	THE EMBED PATH — built
Priya, at Asite host identity	Priya, at Asite known only to Asite
↓ blocked	↓
Annotation.author @ManyToOne User	host.init.identity display name · subjectId · capabilities
↓	↓
OUR users table she must exist as a row	Nothing persisted the viewer stamps no author
Integrating through this API means shadow- provisioning every host user into our database.	The host stays the system of record, and stamps the author itself when it stores the markup.

Figure 3 — Both paths exist in the code. /api/annotations still keys an author to a row in our own user table, and always will for this platform’s own users. The embed path never reaches it: the viewer emits a markup with no author at all, and the host attributes it from the session it already holds.

No external document identity on our own Document. The protocol carries externalId end to end, so an embedding host never needs one — but the platform’s own Document entity still has no such column, so anything integrating through /api/documents keeps a map between its id and our numeric one.

35 endpoints across 15 files — 11 services and 4 components — derived by walking imports transitively from every viewer component, viewer service and viewer-core file. Two groups drive the design: **five are content**, and are exactly what the ingress replaces, and **seventeen are document operations**, which ADR 14 assigned to the host. The remainder is markup, identity and collaboration.

That total has been wrong twice, in the same direction. It was “seven”, then “26 across 9”, and both were undercounts — the first from grepping directories instead of following imports, the second from a resolver that turned `role.service` into `role.ts` by replacing the extension rather than appending one, so every `*.service.ts` import silently failed to resolve and its endpoints were never counted. **A count that resolves imports must fail loudly when one does not resolve**, or it reports the surface it could see as the surface that exists.

And it is not settled yet. `viewer-extraction-inventory.md` §2 gives a total of 35 across 11 services and 4 components, but its own table lists ten services, and its group sizes — content 5, document operations 17, markup 7, identity 2 — sum to 31. Two of those three figures must be wrong. The groups quoted above are the ones the inventory states unambiguously and that a design decision actually turns on; **the partition needs re-deriving before it goes in front of a customer**, and no number here should be quoted as a complete breakdown until it does.

A library in shape, never built here. `viewer-core` has its manifest, its `ng-packagr` configuration and its own `angular.json` project, so `ng build viewer-core` is a real target — it has simply never run in this container, for the Node reason in section 12. The UNLICENSED marker is still deliberate and still means no publish decision has been taken.

Accessibility evidence. The artefacts exist in `cde-platform/docs/accessibility/` — an accessibility statement, a VPAT 2.5 INT conformance report, and a screen-reader matrix — and all three say the same thing about themselves: the statement is a draft not fit to publish, every criterion in the ACR reads Not Evaluated or Does Not Support, and no screen-reader pass has been run. **What is missing is not the document; it is the evidence the document is supposed to cite.** No axe, no Lighthouse budget, no manual keyboard pass, and no gate in the pipeline to produce any of it.

The embed makes this worse rather than inheriting it. Focus crossing a frame boundary, printing from inside a frame, and 200% zoom in a host-sized frame are all new surfaces that need testing afresh, and some of them will be worse than the standalone viewer. §1A is explicit that without a current conformance report the public-sector buyers this product targets cannot be bid to at all — and an independent audit has the longest lead time of anything on this list.

Brand and copyright. The application mark is our own work and deliberately generic; the product has no name or logo that has been through trademark clearance, and LICENSE and NOTICE still carry a placeholder where the copyright holder’s legal entity goes. An obviously unset placeholder is the right state until the entity is decided — a plausible-looking wrong name is a false statement of ownership that survives into every distribution.

The shape of this list has changed since the last issue. The embed and identity contracts were **decisions nobody had taken**; they are taken, specified and implemented. What remains is work nobody has done — and the first item is one response header.